

Guia Completo: Pipeline de Dados End-to-End na AWS

Este documento detalha a implementação técnica de uma esteira de engenharia de dados utilizando Terraform, AWS Glue, Step Functions, Lake Formation e GitHub Actions, seguindo os padrões de mercado para ambientes de missão crítica.

1. Estrutura de Pastas (VSCode)

A organização do repositório é fundamental para a escalabilidade e o uso de CI/CD.

```
projeto-risco-credito/  
├── .github/  
│   └── workflows/           # Esteiras de CI/CD (GitHub Actions)  
├── infra/                   # Código de Infraestrutura (Terraform)  
│   ├── main.tf              # Recursos AWS (S3, Glue, Step Functions)  
│   ├── variables.tf         # Definição de variáveis  
│   ├── outputs.tf           # Saídas do Terraform  
│   ├── iam.tf               # Roles e Políticas  
│   └── lakeformation.tf     # Segurança e Governança  
├── scripts/                 # Código-fonte ETL (PySpark)  
│   └── processar_transacoes.py  
├── tests/                   # Testes unitários  
└── data_sample/             # Amostras de dados para teste  
    └── transacoes_input.json
```

2. Infraestrutura como Código (Terraform)

infra/variables.tf

```
variable "project_name" {  
    default = "itau-risco-credito"  
}  
  
variable "region" {  
    default = "us-east-1"  
}  
  
variable "bucket_prefix" {  
    default = "itau-data-engineering-lab"}
```

```
}
```

infra/iam.tf

```
# Role para o AWS Glue
resource "aws_iam_role" "glue_role" {
  name = "${var.project_name}-glue-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = { Service = "glue.amazonaws.com" }
    }]
  })
}

# Políticas básicas para o Glue
resource "aws_iam_role_policy_attachment" "glue_service" {
  role      = aws_iam_role.glue_role.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole"
}

# Role para o Step Functions
resource "aws_iam_role" "sfn_role" {
  name = "${var.project_name}-sfn-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = { Service = "states.amazonaws.com" }
    }]
  })
}
```

infra/main.tf

```
provider "aws" {
  region = var.region
}

# Buckets S3
resource "aws_s3_bucket" "data_bucket" {
  bucket = "${var.bucket_prefix}-data"
}

resource "aws_s3_bucket" "scripts_bucket" {
  bucket = "${var.bucket_prefix}-scripts"
}
```

```

# Glue Database
resource "aws_glue_catalog_database" "risco_db" {
  name = "db_risco_credito"
}

# Glue Job
resource "aws_glue_job" "process_job" {
  name      = "job-processor-transacoes"
  role_arn = aws_iam_role.glue_role.arn

  command {
    script_location = "s3://${aws_s3_bucket.scripts_bucket.bucket}/scripts/processor-transacoes.py"
    python_version  = "3"
  }

  default_arguments = {
    "--DB_NAME"      = aws_glue_catalog_database.risco_db.name
    "--S3_SOURCE"    = "s3://${aws_s3_bucket.data_bucket.bucket}/raw/"
    "--S3_TARGET"    = "s3://${aws_s3_bucket.data_bucket.bucket}/refined/"
    "--job-language" = "python"
  }
}

# Step Function
resource "aws_sfn_state_machine" "pipeline" {
  name      = "pipeline-risco-credito"
  role_arn = aws_iam_role.sfn_role.arn

  definition = jsonencode({
    StartAt = "RunGlueJob",
    States = {
      RunGlueJob = {
        Type = "Task",
        Resource = "arn:aws:states:::glue:startJobRun.sync",
        Parameters = {
          JobName = aws_glue_job.process_job.name
        },
        End = true
      }
    }
  })
}

```

infra/lakeformation.tf

```

# Registrar o bucket no Lake Formation
resource "aws_lakeformation_resource" "data_location" {
  arn = aws_s3_bucket.data_bucket.arn
}

# Conceder permissões ao Glue Role
resource "aws_lakeformation_permissions" "glue_permissions" {
  principal = aws_iam_role.glue_role.arn
}

```

```
permissions = ["ALL"]

database {
  name = aws_glue_catalog_database.risco_db.name
}
```

3. Script de Processamento (PySpark)

scripts/processar_transacoes.py

```
import sys
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

# Captura de Variáveis de Ambiente do Terraform
args = getResolvedOptions(sys.argv, ['JOB_NAME', 'S3_SOURCE', 'S3_TARGET'])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# 1. Leitura do arquivo JSON (Raw)
df = spark.read.json(args['S3_SOURCE'])

# 2. Transformação simples (ex: adicionar coluna de status)
from pyspark.sql.functions import lit
df_refined = df.withColumn("status_processamento", lit("CONCLUIDO"))

# 3. Escrita em Parquet (Refined)
df_refined.write.mode("overwrite").parquet(args['S3_TARGET'])

job.commit()
```

4. Esteira de CI/CD (GitHub Actions)

.github/workflows/deploy.yml

```
name: Deploy Data Pipeline
on:
  push:
    branches: [ main ]
```

```

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v1
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY }
          aws-secret-access-key: ${ secrets.AWS_SECRET_KEY }
          aws-region: us-east-1

      - name: Sync Scripts to S3
        run: |
          aws s3 sync scripts/ s3://itau-data-engineering-lab-scripts/scripts/

      - name: Terraform Init & Apply
        working-directory: ./infra
        run: |
          terraform init
          terraform apply -auto-approve

```

5. Dataset de Exemplo

data_sample/transacoes_input.json

```

{"id_transacao": "TX-001", "valor": 1200.50, "moeda": "BRL", "data": "2023-11-01"}
{"id_transacao": "TX-002", "valor": 850.00, "moeda": "BRL", "data": "2023-11-01"}
{"id_transacao": "TX-003", "valor": 3200.10, "moeda": "BRL", "data": "2023-11-02"}

```

6. Passo a Passo para Execução

1. **Local:** Crie a estrutura de pastas e os arquivos acima no VSCode.
2. **AWS:** Garanta que você tem permissão de Administrador para o primeiro deploy do Terraform.
3. **GitHub:** Crie os `Secrets` (`AWS_ACCESS_KEY` e `AWS_SECRET_KEY`) no seu repositório.
4. **Deploy:** Faça o `git push`. A esteira irá criar os recursos e subir o script.
5. **Massa de Dados:** Suba o arquivo `transacoes_input.json` para o bucket `...-data/raw/`.

6. **Execução:** No console AWS, acesse o **Step Functions** e inicie a máquina de estados criada.
7. **Athena:** Após a conclusão, vá ao Athena e crie uma tabela apontando para o caminho `...-data/refined/` para consultar os dados Parquet.

Dica Profissional: Em um ambiente real, o Lake Formation exige que você registre a localização do S3 e defina quem são os administradores do Data Lake antes que o Terraform consiga atribuir permissões específicas.